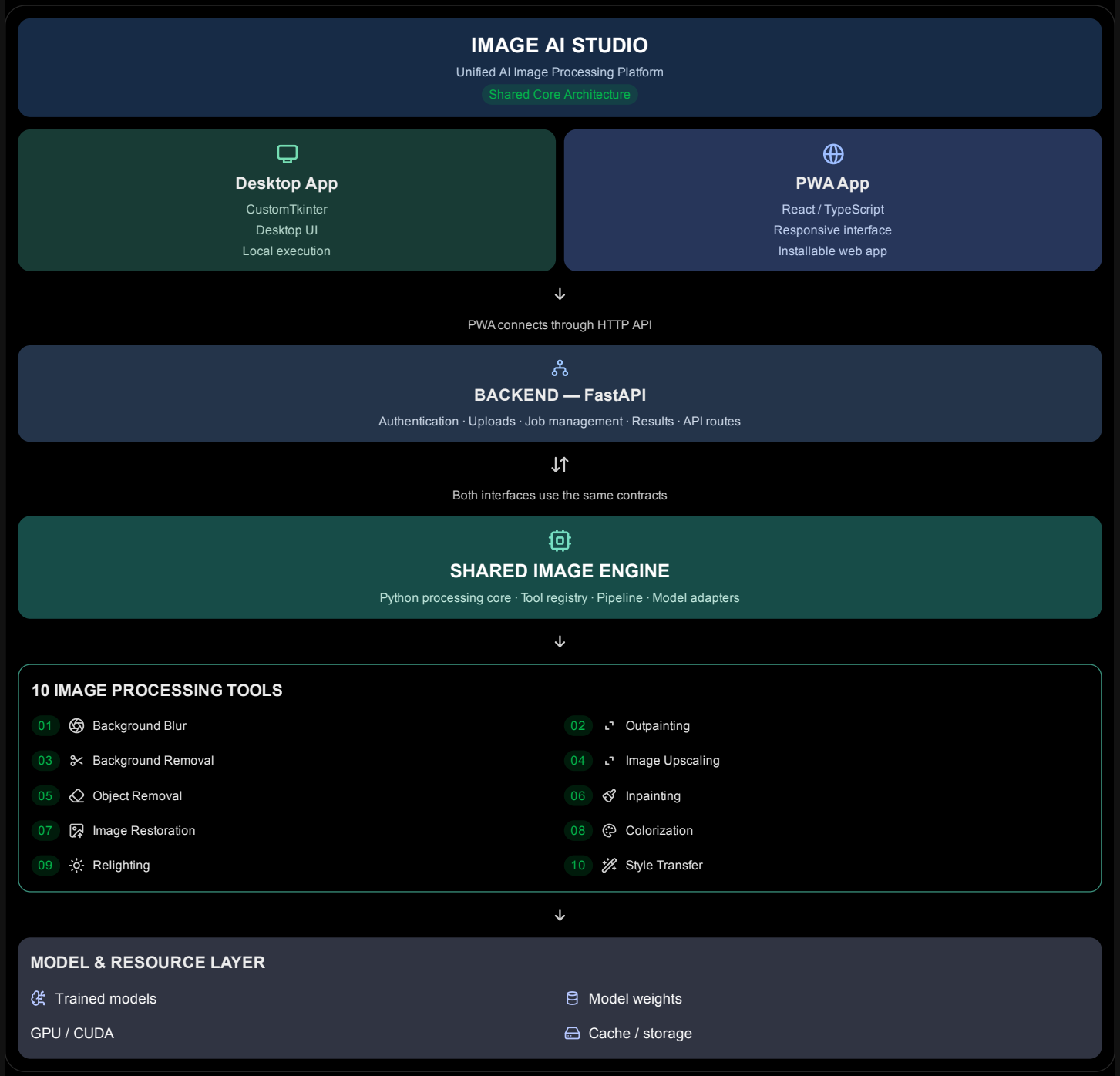
ChatGPT said:

Yes. The best architecture is to build **one shared image-processing engine with two independent interfaces**:

1. **Desktop version:** Python + CustomTkinter.
2. **Web/PWA version:** Frontend + Backend API, both connected to the same engine, models, tools, and processing pipeline.

This avoids duplicating your 10 image tools and allows you to maintain one codebase for the actual AI functionality.

1. Graphic architecture map



2. Recommended project folder architecture

This is how I would organize the actual files.

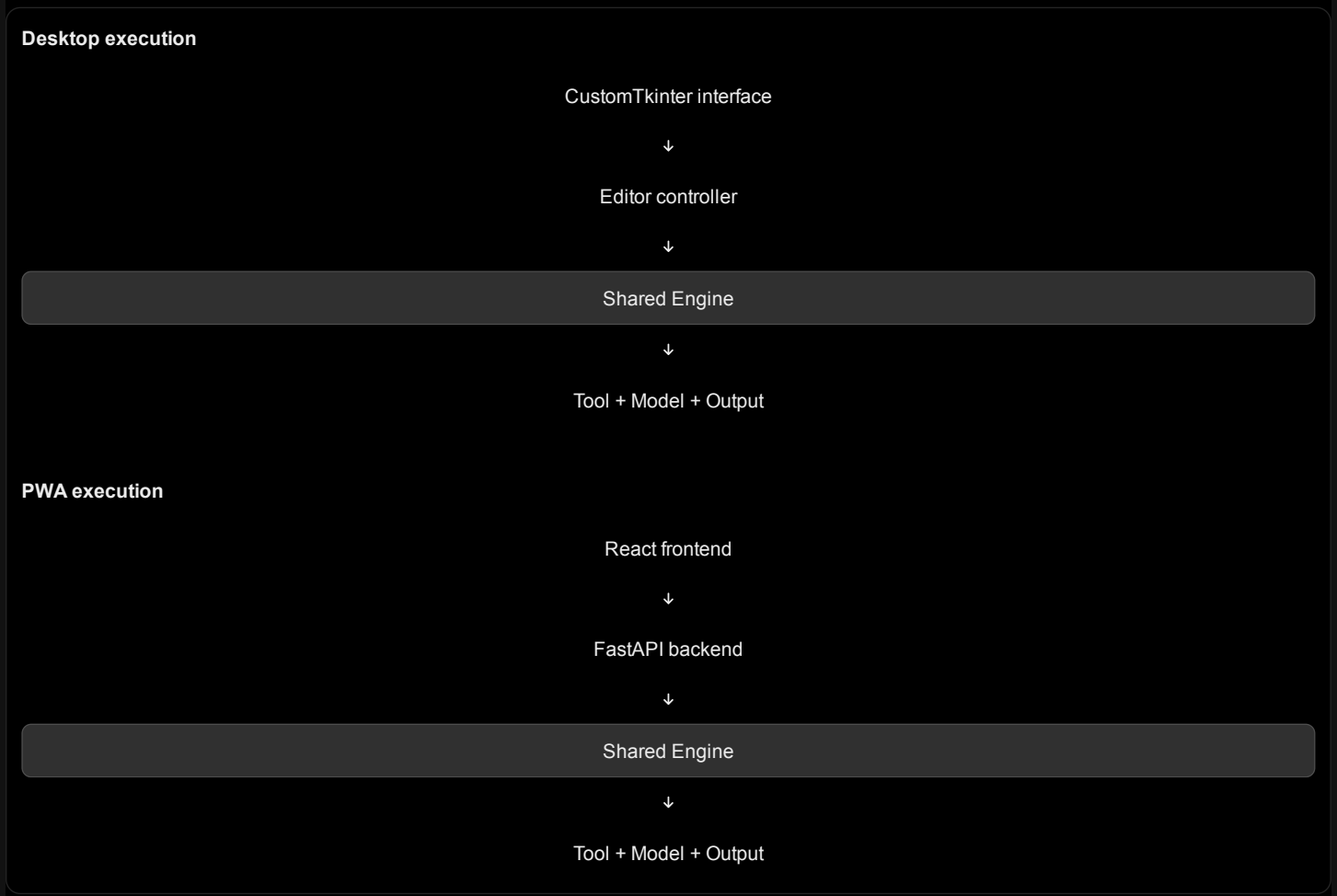
Plain text

```
IMAGE_AI_STUDIO/
│
├── main.py
│
├── config/
│   ├── settings.py
```

```
│   ├── paths.py
│   └── model_config.py
│
├── engine/                    # SHARED CORE
│   ├── __init__.py
│   ├── engine.py
│   ├── tool_registry.py
│   ├── pipeline.py
│   ├── image_loader.py
│   ├── image_exporter.py
│   ├── device_manager.py
│   └── model_manager.py
│
├── tools/                     # 10 IMAGE TOOLS
│   ├── blur/
│   │   ├── tool.py
│   │   └── config.py
│   ├── outpaint/
│   │   ├── tool.py
│   │   └── config.py
│   ├── background_removal/
│   ├── upscaling/
│   ├── object_removal/
│   ├── inpainting/
```

3. How the two versions communicate

The most important architectural decision is that neither interface should contain the actual AI algorithms.



For example, when someone selects Background Blur:

- Desktop calls `engine.run("blur", image, parameters)`.
- PWA sends a request to `/api/tools/blur`.
- FastAPI calls the same `engine.run()` function.
- The engine loads the appropriate model, processes the image and returns the result.

Both versions therefore use the same tool implementation.

4. Important architectural rules

Component	Responsibility
`main.py`	Application launcher
`engine.py`	Central processing engine
`tool_registry.py`	Registers all 10 tools
`model_manager.py`	Loads and manages AI models
`shared/`	Common input/output definitions
`desktop/`	CustomTkinter application
`backend/`	PWA API and server
`frontend/`	Web interface
`storage/`	Images, projects and outputs

Three important decisions:

1. Each tool should be an independent module with its own parameters, defaults, validation and model requirements.
2. Models should be loaded on demand rather than loading all ten models into RAM/VRAM simultaneously.
3. The PWA should execute AI processing on the backend machine/server, not assume that the browser can access Python models directly.

5. Technology stack

Layer	Recommended technology
Desktop UI	CustomTkinter
AI engine	Python
AI models	PyTorch / Hugging Face
Image processing	OpenCV / Pillow
Backend	FastAPI
Frontend	React + TypeScript
PWA	Vite PWA
Communication	REST API + WebSocket
Task processing	Background job queue
Storage	Local filesystem initially

6. Development order

I recommend building it in this order:

1. **Shared engine**

   Create the engine, registry, model manager and common interfaces.

2. **Ten independent tools**

   Implement each tool separately and test its inputs and outputs.

3. **Desktop application**

   Connect CustomTkinter to the engine and build the image editor.

4. **Backend**

   Expose the engine through FastAPI and add asynchronous processing.

5. **PWA frontend**

   Reuse the same API contracts and build the browser-based editor.

6. **Integration and deployment**

   Test both interfaces, GPU access, model loading, uploads and exports.

7. One decision before building the actual project

There is one architectural distinction worth deciding now: how the PWA accesses the AI engine.

1. Where should the PWA run its AI processing?

- (•) Same PC — local FastAPI server, accessible through browser
- ( ) Cloud server — access the AI engine remotely
- ( ) Both — local mode and cloud deployment support

2. How should the trained models be managed?

- ( ) All models stored locally
- (•) Local models + downloadable models when needed
- ( ) Models hosted remotely

3. What kind of interface are you targeting?

- ( ) Full Photoshop-like editor (layers, canvas, masks, history)
- (•) Simple AI tools dashboard with before/after previews
- ( ) Full editor + dedicated workspace for each AI tool

Confirm architecture

My recommendation is **both deployment modes, hybrid model management, and a full editor with dedicated AI tool workspaces**. This gives you a reusable image-editing platform rather than ten disconnected AI scripts.